CMPE3209 Computer Organization — Proje Teslim Tutanağı

Kullanım talimatı

Bu tutanak projenin tek resmi teslim belgesidir. Her kilometre taşında (M1, M2 ve final; tümü pazartesi) ilgili adım doldurulur ve aynı dosya güncellenerek yeniden teslim edilir. Ayrıntılı görev tanımları proje yönergesindedir; bu tutanak yalnızca ne istendiğini ve nereye yazılacağını gösterir.

- Tüm sayılar birimiyle yazılır (byte, cycle, ns, GHz). Hex değerler 0x önekiyle yazılır.
- **Varsayım** olan her değer "(varsayım)" etiketiyle işaretlenir.
- Tablolara sığmayan çıktılar (assembly listesi, objdump, CSV izi, ekran görüntüleri) Git deposuna konur. Tutanakta dosya adıyla atıf yapılır.
- Boş bırakılan zorunlu alan, o adımın puanını sıfırlar.

Öğrenci bilgileri

Alan	Değer
Ad Soyad	Ali Bal
Öğrenci numarası (SID)	5001230006
E-posta	alibal@ogr.iu.edu.tr
Seçilen görev (G1–G5)	G2 - Not dağılımı
Seçilen ISA ve sürümü (ör. RV32IM)	RV32IM / ILP32
Derleyici ve sürümü	Clang 18.1.6 (Zig 0.13.0)
Simülatör / araçlar	Unicorn 2.1.4 Capstone 5.0.9
Git deposu adresi	github.com/AliBal7/ cmpe3209-proje

Adım 0 — Kişiye özel parametreler

Bu adım M1 ile birlikte teslim edilir. Hatalı bir parametre, ona dayanan tüm adımları geçersiz kılar; öğretim üyesi bu tabloyu M1'de kontrol eder.

Sembol	Tanım	Hesap (adımlarıyla)	Sonuç
d	SID'in son hanesi	Son hane	6
D2	SID'in son iki hanesi	Son iki hane	06
S	SID'in hanelerinin toplamı	5+0+0+1+2+3+ 0+0+0+6	17
N	24 + (D2 mod 25)	24+(6 mod 25)	30
Cache	d'ye göre: boyut / line / mapping / line sayısı / set sayısı	64 B / 16 B Direct; 4 line	Set yok

Kod sabiti SID=1230006 (son 9 hane). RV32IM seçimi: sabit 32 bit instruction formatları elle kodlamayı kolaylaştırır; M uzantısı tamsayı çarpma ve bölme içerir. ISA: RV32I 2.1 + M 2.0.

Write policy	d çift → write-back, d tek → write-through	6 çift	write-back
k	(S mod 7) + 3	(17 mod 7)+3	6
Interrupt türü	S çift → Timer, S tek → I/O	17 tek	I/O
f	1.0 + 0.1 × d GHz (varsayım)	1.0+0.1×6	1.6 GHz
T1 / T2	T1 = 1 cycle; T2 = 20 + 5 × (d mod 4) cycle (varsayım)	T1=1 cycle T2=20+5×2	1 / 30 cycle
Nc	2 + (S mod 7) (varsayım)	2+(17 mod 7)	5
Görev parametresi	Seçilen göreve göre (T, P, K, yön veya R)	50+(1230006 mod 10)	P=56

Adres alanları (ISA'nın adres genişliği s + w bit; 64-bit ISA'da düşük 32 bit kullanılır):

Alan	Bit sayısı	Hesap
Word (w)	4	log2(16)
Line (r) veya set (d)	2	log2(4)
Tag (s − r veya s − d)	26	32-4-2
Toplam	32	26+2+4

RV32IM: 32 bit adres. s=28. Frekans, erişim süreleri ve çekirdek sayısı model varsayımıdır.

Adım 1 — Görev seçimi ve C programı (M1: 2 Kasım, Pazartesi)

1.1 Görev seçimi (yalnızca birini işaretleyin):

- ☐ G1 Biyomedikal sinyal alarmı
- ☑ G2 Sınav notu dağılımı
- ☐ G3 Mesaj şifreleme ve bütünlük
- ☐ G4 Sıralama ve medyan
- ☐ G5 Run-length sıkıştırma

1.2 Çözüm tasarımı (3–5 cümle: hangi algoritma, hangi fonksiyonlar, hangi veri yerleşimi ve neden):

Notları LCG ile grades dizisine üretiyorum. İkinci geçişte notları gruplandırıp toplamı ve geçenleri sayıyorum. Ortalamayı bulduktan sonra mark_below ile düşük notları işaretliyorum. Sonuçları global değişkenlerde tutuyorum.

1.3 C kaynak kodu. Satır numaralı kod aşağıdaki bloka yapıştırılır (25–60 satır):

```
01  /* Ali Bal, 5001230006, G2 */
02  const int SID = 1230006;
03  enum { N = 30 };
04  int grades[N];
05  int bucket[5], mean, below_count, pass_count;
06
07  int mark_below(int value, int average)
08  {
09      if (value < average)
10          value = value | 0x100;
11      return value;
12  }
13
14  int main(void)
```

1.3 C kaynak kodu (devam)

```
15  {
16      unsigned int x = (unsigned int)SID;
17      int i, sum = 0, pass = 50 + SID % 10;
18      for (i = 0; i < N; i++) {
19          x = 1664525u * x + 1013904223u;
20          grades[i] = (int)((x >> 16) % 101u);
21      }
22      for (i = 0; i < N; i++) {
23          int v = grades[i];
24          sum = sum + v;
25          if (v < 50) bucket[0]++;
26          else if (v < 65) bucket[1]++;
27          else if (v < 75) bucket[2]++;
28          else if (v < 85) bucket[3]++;
29          else bucket[4]++;
30          if (v >= pass) pass_count++;
31      }
32      mean = sum / N;
33      for (i = 0; i < N; i++) {
34          int v = grades[i];
35          grades[i] = mark_below(v, mean);
36          if (grades[i] & 0x100) below_count++;
37      }
38      return 0;
39  }
```

M1 geri bildirimi uygulandı: 39 satırlık kod artık 1.3 bloğundadır. Onaylanan main.c ve M1 kernel'i değişmedi.

1.4 Zorunlu yapı eşlemesi. Her madde için satır numarası ve tek cümlelik açıklama:

#	Zorunlu yapı	Satır no	Açıklama
Y1	N elemanlı global int dizisi, en az iki geçiş	4,18-37	3 dizi geçişi
Y2	Koşullu dallanma	9,25-30,36	if koşulları
Y3	Parametreli, değer döndüren fonksiyon çağrısı	7-12,35	mark_below
Y4	Toplama/çıkarma, çarpma veya shift, bit işlemi	10,19-24,36	ALU/OR/AND

Y5	Dizi elemanına store	20,35	Diziye store
Y6	const int SID ve hesapta kullanımı	2,16-17	SID hesapta
Y7	Sonucun tutulduğu global değişken(ler)	5,25-32,36	Global sonuçlar

1.5 Program çıktıları. Simülatörde çalıştırıldıktan sonra bellekten okunan değerler:

Global değişken	Adres (hex)	Değer	Ekran görüntüsü dosyası
bucket	0x00020088	{14,1,7,0,8}	evidence/rv32im_simulator_screenshot.jpg (Ek-2)
mean	0x00020008	51	evidence/rv32im_simulator_screenshot.jpg (Ek-2)
below_count	0x0002000C	14	evidence/rv32im_simulator_screenshot.jpg (Ek-2)
pass_count	0x00020004	15	evidence/rv32im_simulator_screenshot.jpg (Ek-2)

1.6 İzlenecek kernel (12–20 dinamik instruction; en az bir load, store, ALU, koşullu branch ve call/return içermeli):

Alan	Değer
C satır aralığı	7-12 ve 35; Ek-2
Başlangıç ve bitiş adresi (hex)	0x000102D4 → 0x0001004C
Dinamik instruction sayısı	20
Hangi döngü iterasyonu (ör. i = 3)	3. döngü; i=0

Öğretim üyesi (M1): ☐ Onaylandı ☐ Düzeltme istendi ☐ Reddedildi — Tarih: _____ Not: _____

Adım 2 — Assembly ve machine code (M2: 9 Kasım, Pazartesi)

2a. C → assembly eşlemesi (-O0; listenin tamamı depoda asm_O0.s):

C satır no	C ifadesi	Assembly satırları	Adres aralığı (hex)
16	LCG başlangıcı	78-79	0x00010068-0x0001006C
19	LCG adımı	102-104,106-108,110,112	0x00010098-0x000100B4
20	grades[i] üretimi	114,116-118,120-123,125,127-130,132	0x000100B8-0x000100EC
35	mark_below ve store	346,348-349,351-353,355,357-358,360	0x000102C4-0x000102EC
Tüm 39 C satırının eşlemesi: Ek-M2.A / m2/c_to_assembly.csv.			

Bölüm	Başlangıç adresi (hex)	Boyut (byte)	İçerik
.text	0x00010000	844 byte	Kod

.data / .bss	0x00020004 (.bss)	152 byte	Global veriler

Von Neumann ilkesi bu adreslerde nasıl görünüyor? (1–2 cümle)

Kod (.text) ve veriler (.rodata/.bss) aynı 32 bit adres uzayındadır. .rodata: 0x00020000, 4 byte (SID). .data oluşmadı (0 byte); sıfır başlangıçlı global veriler .bss içindedir.

2b. -O0 ve -O1 karşılaştırması (kernel için):

Ölçü	-O0	-O1	Farkın nedeni
Kernel'deki instruction sayısı	20	3	Inline; çağrı kalktı
Load sayısı	6	0	Stack okumaları kalktı
Store sayısı	5	0	Stack yazmaları kalktı
Kullanılan register sayısı	5	3	5 -> 3; Ek-M2.C

2c. Elle machine code (en az 10 instruction; x86-64 için 8). Kullanılan tüm formatlar kapsanmalı.

#	Adres (hex)	Assembly	Format	Alanlar (ad = binary)	Machine code (hex)	Bellekteki byte sırası	objdump ile aynı mı?
1	0x00010060	lui a0, 0x12c	U	Ek-M2.B #1	0x0012C537	37 c5 12 00	Simülatör: Evet
2	0x00010064	addi a0, a0, 0x4b6	I	Ek-M2.B #2	0x4B650513	13 05 65 4b	Simülatör: Evet
3	0x000100A4	mul a0, a0, a1	R	Ek-M2.B #3	0x02B50533	33 05 b5 02	Simülatör: Evet
4	0x00010018	lw a0, -0xc(s0)	I	Ek-M2.B #4	0xFF442503	03 25 44 ff	Simülatör: Evet
5	0x00010034	sw a0, -0xc(s0)	S	Ek-M2.B #5	0xFEA42A23	23 2a a4 fe	Simülatör: Evet
6	0x00010020	blt a0, a1, 8	B	Ek-M2.B #6	0x00B54463	63 44 b5 00	Simülatör: Evet
7	0x00010028	j 4	J	Ek-M2.B #7	0x0040006F	6f 00 40 00	Simülatör: Evet
8	0x000102D0	auipc ra, 0	U	Ek-M2.B #8	0x00000097	97 00 00 00	Simülatör: Evet
9	0x000102D4	jalr ra, ra, -0x2d0	I	Ek-M2.B #9	0xD30080E7	e7 80 00 d3	Simülatör: Evet
10	0x00010030	ori a0, a0, 0x100	I	Ek-M2.B #10	0x10056513	13 65 05 10	Simülatör: Evet

Zorunlu instruction'ların tablodaki sıra numarası:

- ☐ SID'i yükleyen instruction(lar): # 1, 2
- ☐ Koşullu branch: # 6
- ☐ Load: # 4
- ☐ Store: # 5
- ☐ Fonksiyon çağrısı: # 8, 9

SID'in yüklenmesi. Sabitin neden tek instruction'a sığmadığı, nasıl bölündüğü ve gerekiyorsa +1 düzeltmenin sign extension ile açıklaması:

SID=1230006=0x0012C4B6. Üst=(SID+0x800)>>12=0x12C; alt=SID-(0x12C<<12)=0x4B6=1206. Alt alanın işaret biti 0; bu SID için +1 düzeltmesi gerekmiyor. Alt 12 bit >=0x800 olsaydı üst +1, alt negatif olurdu (Ek-M2.C).

Branch offset hesabı. Branch'in adresi, hedef adres ve offset'in adım adım hesabı:

blt adresi=0x00010020, hedef=0x00010028. Offset=hedef-PC=0x8=8 byte; PC+4 değil, instruction adresi esas alınır. imm[12|10:5|4:1|11]=0|000000|0100|0, imm[0]=0. Sonuç: 0x00B54463; little-endian: 63 44 B5 00.

Adım 3 — CPU, interrupt ve bus (final paketiyle: 16 Kasım, Pazartesi)

Model varsayımları (okudum ve uyguladım):

- ☐ Pipeline yok, her an tek instruction yürütülür.
- ☐ PC, IR, MAR ve MBR Fig. 3.2'deki gibi kavramsal olarak vardır.
- ☐ Her bellek erişimi bir bus işlemidir; cache bu adımda yok sayılır.
- ☐ Data bus genişliği = ____ bit.

Modelin gerçek donanımdan ayrıldığı yerler (3–5 cümle):

3a. Instruction cycle izleme tablosu. Her dinamik instruction bir satırdır. Bir instruction birden çok bellek erişimi yapıyorsa MAR, MBR ve bus hücrelerine sırayla yazılır. Kategori: PM = processor–memory, PIO = processor–I/O, DP = data processing, C = control.

#	PC (önce → sonra)	Assembly	IR (hex)	Fig. 3.6 durumları	MAR	MBR	ALU işlemi	Değişen register	Bus (adres / data / control)	Kategori
1										
2										
3										
4										
5										
6										
7										
8										
9										
10										
11										
12										
13										
14										
15										

16										
17										
18										
19										
20										

Kernel toplamları	Sayı
Instruction fetch	
Operand fetch (of)	
Operand store (os)	
PC'nin sıra dışı değiştiği instruction	

3b. Fig. 3.4 makinesine aktarım. Kernel'in tek bir dizi elemanını güncelleyen parçası, opcode 1, 2, 3, 5 ve 7 ile yeniden yazılır ve Fig. 3.5 formatında izlenir:

Adım	Fetch / execute	PC	IR (hex)	AC	Değişen bellek hücresi
1					
2					
3					
4					
5					
6					
7					
8					

Ölçü	Seçilen ISA	Fig. 3.4 makinesi
Aynı iş için instruction cycle sayısı		

Tek accumulator'ın getirdiği sınırlama (PDP-11 `ADD B,A` ile karşılaştırarak):

IAS register'ları ve seçilen ISA:

IAS register'ı	IAS'taki görevi	Seçilen ISA'daki karşılığı	Programcıya görünür mü?
MBR			

MAR			
IR			
IBR			
PC			
AC			
MQ			

3c. Interrupt senaryosu (k = ____, tür = ____):

Soru	Cevap
Interrupt hangi instruction'ın execute'u sırasında geliyor? (adres, assembly)	
Ne zaman tanınır ve bu instruction neden yarıda kesilmez? (Fig. 3.9)	
Kaydedilen PC değeri (hex)	
Kaydedilen diğer durum bilgisi	
Seçilen ISA'daki gerçek mekanizma (register adları, vektör)	
Kaynak (ISA referansı, bölüm, sayfa)	
Handler dönüşünde ilk çalışan instruction'ın adresi	

Fonksiyon çağrısı (Y3) ve interrupt karşılaştırması:

Ölçüt	Fonksiyon çağrısı	Interrupt
Akışı kim başlatır?		
Dönüş adresini kim kaydeder, nereye?		
User program bundan haberdar mı?		

İkinci (daha yüksek öncelikli) interrupt. Aynı olay dizisini iki yaklaşımla zaman çizelgesi olarak doldurun (Fig. 3.13 ve 3.14):

Zaman	Sequential (disable)	Nested (priority)
t_0: 1. interrupt gelir		
t_1: 2. interrupt gelir		
t_2		
t_3		
t_4: user program'a dönüş		

3d. Bus ve interconnection.

Bus işlemi türü (kernel, cache yok)	Sayı	Aktif control line'lar
Instruction fetch (read)		
Data read		
Data write		
Toplam		

Hesap	Değer	Adımlar
Address bus genişliği ve maksimum adreslenebilir bellek		
Data bus 16 bit olsaydı kernel'in bus cycle sayısı		
Mevcut / 16-bit oranı		

PCIe paket verimi (N × 4 = ____ byte'lık tek Memory Write TLP; Fig. 3.25):

Alan (octet)	12-byte header, ECRC yok	16-byte header, ECRC var
STP framing (baş)		
Sequence number		
Header		
Data		
ECRC		
LCRC		
STP framing (son)		
Toplam		
Verim = data / toplam		
128b/130b ile verim		

Sonucun bir çekirdekten PCIe cihazına yolu (Fig. 3.17 ve 3.21): her adımda bağlantı, protokol katmanı ve aktarım birimi.

Adım 4 — Cache simülasyonu (final paketiyle: 16 Kasım, Pazartesi)

4a. Data adres izi. Tüm load ve store'lar depoda `trace.csv` dosyasındadır. Biçim: `sira,RW,adres_hex,boyut,c_satiri,stack_mi`.

Ölçü	Değer
Kullanılan derleme (-O0 / -O1)	

Toplam data referansı	
Load / store	
Stack erişimi sayısı	
Ana diziye erişim sayısı	

4b. İlk 5 farklı adresin alanları:

Adres (hex)	Adres (binary)	Tag	Line / Set	Word

4c. Elle simülasyon (ilk 30 referans):

#	R/W	Adres (hex)	Tag	Line / Set	Word	Hit / Miss	Yerinden çıkan blok	Bu blok daha önce cache'te miydi?
1								
2								
3								
4								
5								
6								
7								
8								
9								
10								
11								
12								
13								
14								
15								

16								
17								
18								
19								
20								
21								
22								
23								
24								
25								
26								
27								
28								
29								
30								

Cache'in 30. referanstan sonraki anı. Line sayınız 8'den fazlaysa satır ekleyin.

Line	Set	Valid	Tag (hex)	Dirty (write-back)	LRU sırası (associative)
0					
1					
2					
3					
4					
5					
6					
7					

4d. Tam simülasyon sonuçları:

Ölçü	Elle (ilk 30)	Araç (ilk 30)	Araç (tüm iz)
Referans	30	30	
Hit			

Miss			
Hit ratio H			
Ana belleğe yazma sayısı			

Kullanılan araç ve ayarları (kendi simülatörünüz varsa dosya adı):

- ☐ Elle ve araçla yapılan simülasyon ilk 30 referansta birebir örtüşüyor.

4e. Tasarım uzayı (cache boyu sabit):

Mapping (line boyu sabit = ____ B)	Hit	Miss	H
Direct			
2-way set-associative			
4-way set-associative			
Fully associative (LRU)			

Line boyu (mapping sabit = ____)	Hit	Miss	H
8 B			
16 B			
32 B			
64 B			

Grafik dosyası ve eğilimlerin locality ile açıklaması (thrashing varsa çatışan adresler):

4f. Ortalama erişim süresi: $T_s = H \times T1 + (1 - H) \times (T1 + T2)$

Durum	H	T_s (cycle)	Speedup (cache'siz duruma göre)
Cache yok	0	T2 =	1
Kişiye özel cache			

4g. Kod değişikliği:

	Önce	Sonra
Değiştirilen C satır(lar)ı		
Y7 çıktısı (aynı olmalı)		
H		

T_s (cycle)		

Değişikliğin gerekçesi; H artmadıysa nedeni:

Adım 5 — Performans ve architecture/organization (final paketiyle: 16 Kasım, Pazartesi)

5a-1. Dinamik instruction dağılımı ve CPI (CPI_i değerleri yönergedeki model varsayımıdır):

Sınıf	CPI_i (varsayım)	I_i (-O0)	I_i (-O1)
ALU	1		
Çarpma	3		
Load	1		
Store	1		
Branch / jump	2		
Ic (toplam)			
Ortalama CPI			

5a-2. Süre ve MIPS rate (f = ____ GHz, τ = ____ ns):

Ölçü	Formül	-O0	-O1
T (bellek etkisi yok)	$Ic \times CPI \times \tau$		
MIPS rate	$f / (CPI \times 10^6)$		
m (instruction başına data referansı)	data ref. / Ic		
T, cache yok (k = T2)	$Ic \times [p + (m \times k)] \times \tau$		
T, cache var (k = T_s)	$Ic \times [p + (m \times k)] \times \tau$		

Karşılaştırma	Değer
Speedup -O0 → -O1 (Eq. 2.1, cache var)	
Speedup cache yok → cache var (-O0)	

MIPS rate'in iki derlemeyi karşılaştırmada neden yanıltıcı olabileceği, kendi sayılarınızla:

5a-3. Amdahl yasası (Nc = ____):

Alan	Değer
Paralelleştirilebilen kısım (C satırları)	

Bağımsızlık gerekçesi (veri bağımlılığı)	
f (yürütme süresindeki pay)	
Speedup = 1 / [(1 − f) + f / Nc]	
Nc → ∞ limiti	

5b. Architecture ve organization (en az 10 özellik):

#	Özellik	Architecture / Organization	Gerekçe (Stallings tanımı)	Değişseydi Y7 değişir miydi? Kanıt
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				

Ch. 1.2'deki dört temel fonksiyon:

Fonksiyon	Kernel'den örnek instruction (adres, assembly)	Açıklama
Data processing		
Data storage		
Data movement		
Control		

Final teslim (16 Kasım Pazartesi; sözlü savunma 20 Kasım Cuma)

Teslim kontrol listesi:

- ☐ Bu tutanak, tüm adımları dolu olarak (PDF ve düzenlenebilir sürüm)
- ☐ C kaynak kodu (`main.c`) ve sonraki 4g sürümü (`main_v2.c`)
- ☐ `asm_O0.s`, `asm_O1.s` ve `objdump -d` çıktısı
- ☐ `trace.csv` ve cache simülasyon çıktıları (kendi simülatörünüz varsa kaynak kodu)
- ☐ Ekran görüntüleri: machine code doğrulaması, program çıktıları, cache durumu

- ☐ 4e grafiği
- ☐ Git deposunda M1, M2 ve final etiketleri

Kilometre taşı kaydı (öğretim üyesi doldurur):

Kilometre taşı	Son tarih	Teslim tarihi	Git etiketi	Paraf
M1 Öneri (Adım 0–1)	2 Kasım (Pzt)			
M2 Assembly ve machine code (Adım 2)	9 Kasım (Pzt)			
M3 CPU, interrupt, bus (Adım 3)	16 Kasım (Pzt, final paketiyle)			
M4 Cache ve performans (Adım 4–5)	16 Kasım (Pzt, final paketiyle)			
Final ve sözlü savunma	Teslim 16 Kasım (Pzt); savunma 20 Kasım (Cuma)			

YZ kullanım beyanı. Kullanılmadıysa ilk satıra "Kullanılmadı" yazın. Beyan edilmemiş kullanım akademik dürüstlük ihlalidir.

Araç ve sürümü	Adım	Ne için kullanıldı	Örnek prompt	Çıktıyı nasıl doğruladım
ChatGPT/ Codex	M1	Kod denetimi; derleme hatalarının giderilmesi.	“M1 kısmını tamamlamamız lazım.”	RV32IM yürütmesi ve bağımsız hesap karşılaştırıldı.
ChatGPT/ Codex	M2	Kodlama ve derleme denetimi; hata düzeltme.	“M2 deposunu da tamamlayıp…”	ELF bit/byte eşlemesi; O0/O1 gerçek yürütmesi; bağımsız referans.

Öğrenci beyanı. Bu tutanaktaki C kodunu, elle machine code kodlamasını, instruction cycle izleme tablosunu ve elle cache simülasyonunu kendim yaptım. Başka bir öğrencinin çalışmasını kopyalamadım ve kendi çalışmamı paylaşmadım. YZ kullanımımı eksiksiz beyan ettim. Sözlü savunmada bu çalışmayı açıklayamamamın puanımı etkileyeceğini biliyorum.

Ad Soyad	Öğrenci numarası	Tarih	İmza
Ali Bal	5001230006		

Değerlendirme (öğretim üyesi doldurur)

Bileşen	Azami	Puan	Not
A1 Öneri ve parametre hesabı (Adım 0–1)	5		
A2 Assembly ve machine code (Adım 2)	15		
A3 CPU, interrupt, bus (Adım 3)	25		
A4 Cache (Adım 4)	25		
A5 Performans ve architecture/organization (Adım 5)	10		

Tutanak ve rapor kalitesi	5		
Sözlü savunma	15		
Geç teslim kesintisi	–		
Toplam	**100**		

Sözlü savunma kaydı:

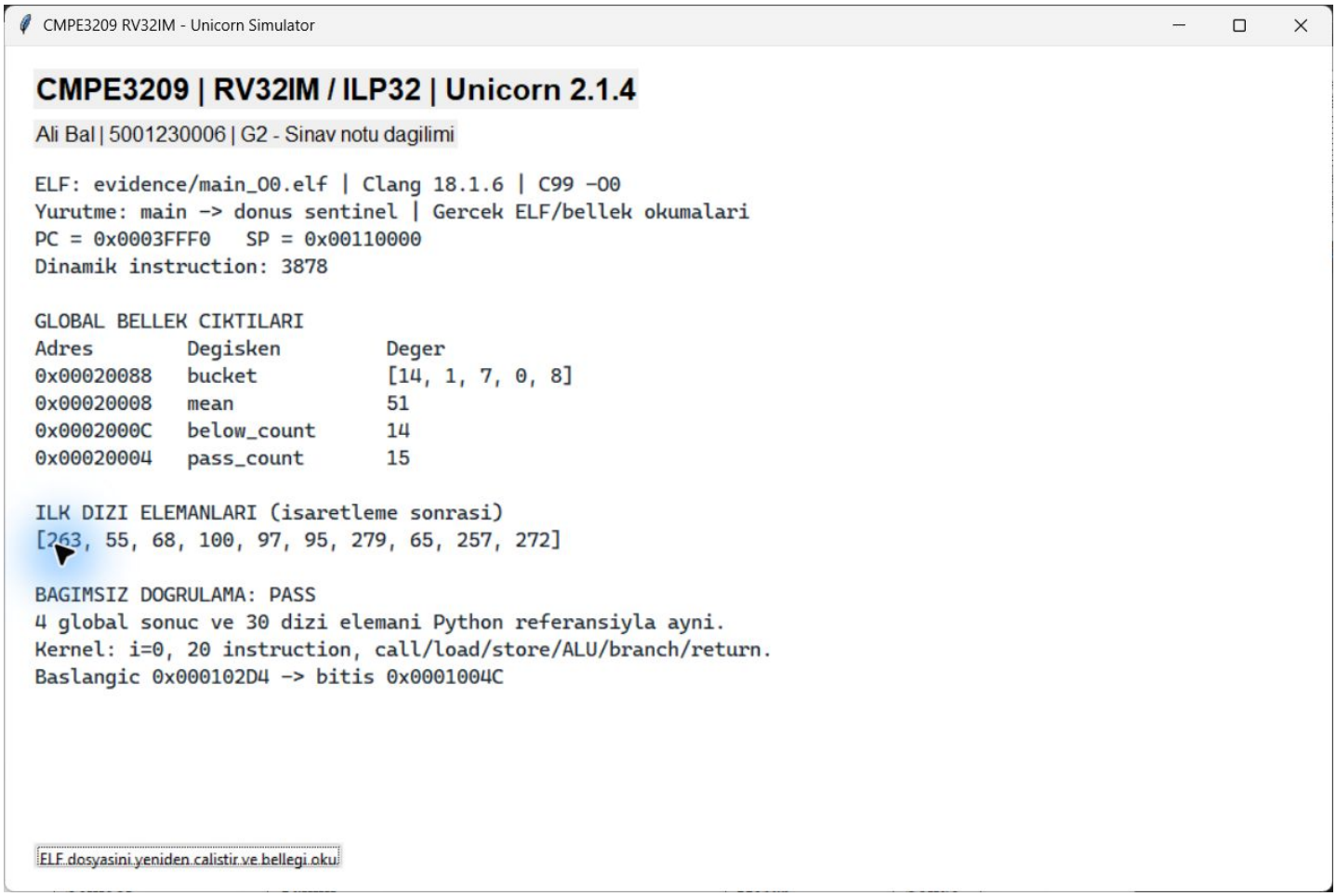
Madde	Sorulan	Sonuç (başarılı / kısmen / başarısız)	Puan
1. Canlı encode (-O1 listesinden)			
2. Adres alanları ve hit/miss			
3. Interrupt k+1'de gelseydi			
4. Tasarım kararı savunması			

☐ 1. ve 2. maddelerin ikisi de başarısız: A2–A4 puanları 0.5 ile çarpıldı.

Öğretim üyesi	Tarih	İmza

Ek-2 - Simülatör çıktısı ve izlenecek kernel

Ali Bal | 5001230006 | G2 | RV32IM | Unicorn 2.1.4



Gerçek simülatör penceresi: ELF yeniden çalıştırılarak global bellek değerleri okunmuştur. Sonuçlar bağımsız Python hesabıyla karşılaştırılmıştır.

Kernel: C 7-12 ve 35, üçüncü döngü i=0. v=7, mean=51; dönüş 263 (0x107). Başlangıç 0x000102D4, bitiş 0x0001004C. Adresler dinamik yürütme sırasındadır. Store'lar stack'e yapılır; ana diziye store çağrıdan sonra gelir.

```
01  0x000102D4  jalr  ra, ra, -0x2d0
02  0x00010000  addi  sp, sp, -0x10
03  0x00010004  sw    ra, 0xc(sp)
04  0x00010008  sw    s0, 8(sp)
05  0x0001000C  addi  s0, sp, 0x10
06  0x00010010  sw    a0, -0xc(s0)
07  0x00010014  sw    a1, -0x10(s0)
08  0x00010018  lw    a0, -0xc(s0)
09  0x0001001C  lw    a1, -0x10(s0)
10  0x00010020  blt   a0, a1, 8
11  0x00010028  j     4
12  0x0001002C  lw    a0, -0xc(s0)
13  0x00010030  ori   a0, a0, 0x100
14  0x00010034  sw    a0, -0xc(s0)
15  0x00010038  j     4
16  0x0001003C  lw    a0, -0xc(s0)
17  0x00010040  lw    ra, 0xc(sp)
18  0x00010044  lw    s0, 8(sp)
19  0x00010048  addi  sp, sp, 0x10
20  0x0001004C  ret
```

20 instruction: call/return, lw, sw, addi/ori ve blt içerir. Ayrıntılı veri: evidence/verification.json.

Ek-M2.A - C / assembly / ELF eşlemesi

Tüm C satırları aşağıdadır. Assembly satır numaraları kökteki asm_O0.s dosyasına aittir. ELF adresleri .debug_line (DWARF) üzerinden gerçek 32 bit komutlara eşlenmiştir. Aynı C satırı birden fazla komut aralığı üretebilir.

Assembler sonrası bazı branch'ler ters koşul + jal olarak açılmıştır. Bu yüzden .s satır sayısı ile ELF instruction sayısı aynı değildir. Kesin makine komutları m2/disassembly_O0.txt içindedir; disassembler Capstone 5.0.9'dur.

C no	C ifadesi / açıklama	asm_O0.s satırları	ELF komut adresleri
1	/* Ali Bal, 5001230006, G2 */ Bağımsız yürütülebilir instruction yok (bildirim/etiket/yorum/parantez).	-	-
2	const int SID = 1230006; SID: .rodata; ayrıca C16 başlangıcı için lui/addi ile sabit katlanır.	-	-
3	enum { N = 30 }; N=30 derleme zamanı sabiti; döngü limitlerine gömülür.	-	-
4	int grades[N]; Global diziler/sonuçlar .bss içindedir; başlangıç sıfırdır.	-	-
5	int bucket[5], mean, below_count, pass_count; Global diziler/sonuçlar .bss içindedir; başlangıç sıfırdır.	-	-
6	(boş satır) Bağımsız yürütülebilir instruction yok (bildirim/etiket/yorum/parantez).	-	-
7	int mark_below(int value, int average) Bağımsız yürütülebilir instruction yok (bildirim/etiket/yorum/parantez).	-	-
8	{	15,17-18,21,23-24	0x00010000-0x00010014
9	if (value < average)	27,29,32-33	0x00010018-0x00010028
10	value = value | 0x100;	37,39,41,43	0x0001002C-0x00010038
11	return value;	47-49,51	0x0001003C-0x0001004C
12	} Bağımsız yürütülebilir instruction yok (bildirim/etiket/yorum/parantez).	-	-
13	(boş satır) Bağımsız yürütülebilir instruction yok (bildirim/etiket/yorum/parantez).	-	-
14	int main(void) Bağımsız yürütülebilir instruction yok (bildirim/etiket/yorum/parantez).	-	-
15	{	66,68-69,72,74-75	0x00010050-0x00010064
16	unsigned int x = (unsigned int)SID;	78-79	0x00010068-0x0001006C
17	int i, sum = 0, pass = 50 + SID % 10;	81-82,84	0x00010070-0x00010078
18	for (i = 0; i < N; i++) {	87,89,93-94,97-98,138-140,142	0x0001007C-0x00010094; 0x000100F4-0x00010100
19	x = 1664525u * x + 1013904223u;	102-104,106-108,110,112	0x00010098-0x000100B4
20	grades[i] = (int)((x >> 16) % 101u);	114,116-118,120-123,125,127-130,132	0x000100B8-0x000100EC
21	}	134	0x000100F0
22	for (i = 0; i < N; i++) {	149,151,155-156,159-160,297-299,301	0x00010108-0x00010120; 0x0001024C-0x00010258
23	int v = grades[i];	164,166-170,172	0x00010124-0x0001013C

C no	C ifadesi / açıklama	asm_O0.s satırları	ELF komut adresleri
24	sum = sum + v;	174,176,178,180	0x00010140-0x0001014C
25	if (v < 50) bucket[0]++;	183-184,187-188,192-195,197	0x00010150-0x00010174
26	else if (v < 65) bucket[1]++;	201-202,205-206,210-214,216	0x00010178-0x000101A0
27	else if (v < 75) bucket[2]++;	220-221,224-225,229-233,235	0x000101A4-0x000101CC
28	else if (v < 85) bucket[3]++;	239-240,243-244,248-252,254	0x000101D0-0x000101F8
29	else bucket[4]++;	257-262	0x000101FC-0x00010210
30	if (v >= pass) pass_count++;	274,276,279-280,284-287,289	0x00010220-0x00010244
31	}	293	0x00010248
32	mean = sum / N;	305-307,309-313,315-317	0x0001025C-0x00010284
33	for (i = 0; i < N; i++) {	320,322,326-327,330-331,390-392,394	0x00010288-0x000102A0; 0x00010328-0x00010334
34	int v = grades[i];	335,337-342,344	0x000102A4-0x000102C0
35	grades[i] = mark_below(v, mean);	346,348-349,351-353,355,357-358,360	0x000102C4-0x000102EC
36	if (grades[i] & 0x100) below_count++;	363,365-366,368-369,372-373,377-380,382	0x000102F0-0x00010320
37	}	386	0x00010324
38	return 0;	402	0x00010344-0x00010348
39	} Bağımsız yürütülebilir instruction yok (bildirim/etiket/yorum/parantez).	-	-

Ek-M2.B - Instruction kodlamaları 1-2

#1 | 0x00010060 | lui a0, 0x12c | U formatı

SID üst 20 bit. Semantik girdiler: rd=10, imm=300

Instruction bitleri	Alan	Binary değer
31:12	imm[31:12]	`00000000000100101100`
11:7	rd	`01010`
6:0	opcode	`0110111`

`32 bit word: 00000000000100101100010100110111`

Hex: 0x0012C537 | Bellekte düşük adresten yükseğe: 37 c5 12 00 | ELF / simülatör: AYNI

#2 | 0x00010064 | addi a0, a0, 0x4b6 | I formatı

SID alt 12 bit. Semantik girdiler: rd=10, rs1=10, imm=1206

Instruction bitleri	Alan	Binary değer
31:20	imm[11:0]	`010010110110`
19:15	rs1	`01010`
14:12	funct3	`000`
11:7	rd	`01010`
6:0	opcode	`0010011`

`32 bit word: 01001011011001010000010100010011`

Hex: 0x4B650513 | Bellekte düşük adresten yükseğe: 13 05 65 4b | ELF / simülatör: AYNI

Ek-M2.B - Instruction kodlamaları 3-4

#3 | 0x000100A4 | mul a0, a0, a1 | R formatı

LCG çarpması. Semantik girdiler: rd=10, rs1=10, rs2=11

Instruction bitleri	Alan	Binary değer
31:25	funct7	`0000001`
24:20	rs2	`01011`
19:15	rs1	`01010`
14:12	funct3	`000`
11:7	rd	`01010`
6:0	opcode	`0110011`

`32 bit word: 00000010101101010000010100110011`

Hex: 0x02B50533 | Bellekte düşük adresten yükseğe: 33 05 b5 02 | ELF / simülatör: AYNI

#4 | 0x00010018 | lw a0, -0xc(s0) | I formatı

Notu stack üzerinden yükle. Semantik girdiler: rd=10, rs1=8, imm=-12

Instruction bitleri	Alan	Binary değer
31:20	imm[11:0]	`111111110100`
19:15	rs1	`01000`
14:12	funct3	`010`
11:7	rd	`01010`
6:0	opcode	`0000011`

`32 bit word: 11111111010001000010010100000011`

Hex: 0xFF442503 | Bellekte düşük adresten yükseğe: 03 25 44 ff | ELF / simülatör: AYNI

Ek-M2.B - Instruction kodlamaları 5-6

#5 | 0x00010034 | sw a0, -0xc(s0) | S formatı

İşaretli notu stack'e yaz. Semantik girdiler: rs1=8, rs2=10, imm=-12

Instruction bitleri	Alan	Binary değer
31:25	imm[11:5]	`1111111`
24:20	rs2	`01010`
19:15	rs1	`01000`
14:12	funct3	`010`
11:7	imm[4:0]	`10100`
6:0	opcode	`0100011`

`32 bit word: 11111110101001000010101000100011`

Hex: 0xFEA42A23 | Bellekte düşük adresten yükseğe: 23 2a a4 fe | ELF / simülatör: AYNI

#6 | 0x00010020 | blt a0, a1, 8 | B formatı

Ortalamanın altı: branch. Semantik girdiler: rs1=10, rs2=11, imm=8

Instruction bitleri	Alan	Binary değer
31	imm[12]	`0`
30:25	imm[10:5]	`000000`
24:20	rs2	`01011`
19:15	rs1	`01010`
14:12	funct3	`100`
11:8	imm[4:1]	`0100`
7	imm[11]	`0`
6:0	opcode	`1100011`

`32 bit word: 00000000101101010100010001100011`

Hex: 0x00B54463 | Bellekte düşük adresten yükseğe: 63 44 b5 00 | ELF / simülatör: AYNI

Ek-M2.B - Instruction kodlamaları 7-8

#7 | 0x00010028 | j 4 | J formatı

j sözde komutu: jal x0,+4. Semantik girdiler: rd=0, imm=4

Instruction bitleri	Alan	Binary değer
31	imm[20]	`0`
30:21	imm[10:1]	`0000000010`
20	imm[11]	`0`
19:12	imm[19:12]	`00000000`
11:7	rd	`00000`
6:0	opcode	`1101111`

`32 bit word: 00000000010000000000000001101111`

Hex: 0x0040006F | Bellekte düşük adresten yükseğe: 6f 00 40 00 | ELF / simülatör: AYNI

j +4 sözde komutunun gerçek biçimi jal x0,+4'tür. Dönüş adresi x0'a yazıldığı için saklanmaz.

#8 | 0x000102D0 | auipc ra, 0 | U formatı

call çiftinin üst kısmı. Semantik girdiler: rd=1, imm=0

Instruction bitleri	Alan	Binary değer
31:12	imm[31:12]	`00000000000000000000`
11:7	rd	`00001`
6:0	opcode	`0010111`

`32 bit word: 00000000000000000000000010010111`

Hex: 0x00000097 | Bellekte düşük adresten yükseğe: 97 00 00 00 | ELF / simülatör: AYNI

Ek-M2.B - Instruction kodlamaları 9-10

#9 | 0x000102D4 | jalr ra, ra, -0x2d0 | I formatı

mark_below çağrısı. Semantik girdiler: rd=1, rs1=1, imm=-720

Instruction bitleri	Alan	Binary değer
31:20	imm[11:0]	`110100110000`
19:15	rs1	`00001`
14:12	funct3	`000`
11:7	rd	`00001`
6:0	opcode	`1100111`

`32 bit word: 11010011000000001000000011100111`

Hex: 0xD30080E7 | Bellekte düşük adresten yükseğe: e7 80 00 d3 | ELF / simülatör: AYNI

#10 | 0x00010030 | ori a0, a0, 0x100 | I formatı

8. biti set et. Semantik girdiler: rd=10, rs1=10, imm=256

Instruction bitleri	Alan	Binary değer
31:20	imm[11:0]	`000100000000`
19:15	rs1	`01010`
14:12	funct3	`110`
11:7	rd	`01010`
6:0	opcode	`0010011`

`32 bit word: 00010000000001010110010100010011`

Hex: 0x10056513 | Bellekte düşük adresten yükseğe: 13 65 05 10 | ELF / simülatör: AYNI

Ek-M2.C - Optimizasyon, sabit ve branch hesabı

Karşılaştırmanın sınırı

Aynı mantıksal işlem, i=0 için giriş notu 7 ve ortalama 51 hazırken başlar; işaretli sonuç 263 hazır olduğunda biter. -O0 tarafında M1'de onaylanan 20 instruction'lık pencere aynen kullanılmıştır. -O1 tarafında bu işlem main içine gömülen slt / slli / or üçlüsüdür. Ana diziye load/store her iki mantıksal pencerenin dışındadır.

M1 penceresi jalr instruction'ıyla başlar; hemen önceki auipc hazırlığı bu 20 sayısına dahil değildir. Call çiftinin ikisi de sayılsaydı -O0 için 21 instruction olurdu. -O1'de call ve ret yürütülmez. Sayılar aynı C işleminin gerçek dinamik yürütmesidir; eşit uzunlukta fiziksel bloklar değildir.

O0: 20 instruction, 6 load, 5 store, 5 register (a0, a1, ra, s0, sp). O1: 3 instruction, 0 load, 0 store, 3 register (a0, a5, a6). zero/x0 sayılmamıştır. Register allocation stack trafiğini kaldırır; inlining çağrı/epilogue maliyetini, if-conversion da branch'i kaldırır.

O1 kernel adresleri: 0x00010178 slt a6,a5,a0; 0x0001017C slli a6,a6,8; 0x00010180 or a5,a5,a6. O0 kernel: 0x000102D4 jalr ile başlar, 0x0001004C ret ile biter.

Tüm program sonuçları

O0: 3878 dinamik instruction, 988 load, 502 store; 211 statik instruction, .text 844 byte. O1: 1030 dinamik instruction, 70 load, 123 store; 109 statik instruction, .text 436 byte. Her iki programda bucket={14,1,7,0,8}, mean=51, below_count=14, pass_count=15. İşaretli 30 elemanın tamamı da aynıdır.

SID ve sign extension

SID=1230006=0x0012C4B6. lui a0,0x12C üst kısmı 0x0012C000 yapar; addi a0,a0,0x4B6 buna 1206 ekler. U=(SID+0x800)>>12, L=SID-(U<<12). Bu SID'de L pozitif olduğundan +1 düzeltmesi yoktur. Genel örnek: 0x00123ABC için U=0x124, L=-1348; 0x00124000-1348=0x00123ABC. Üstü artırmak, alt 12 bitin işaret genişletmesini telafi eder.

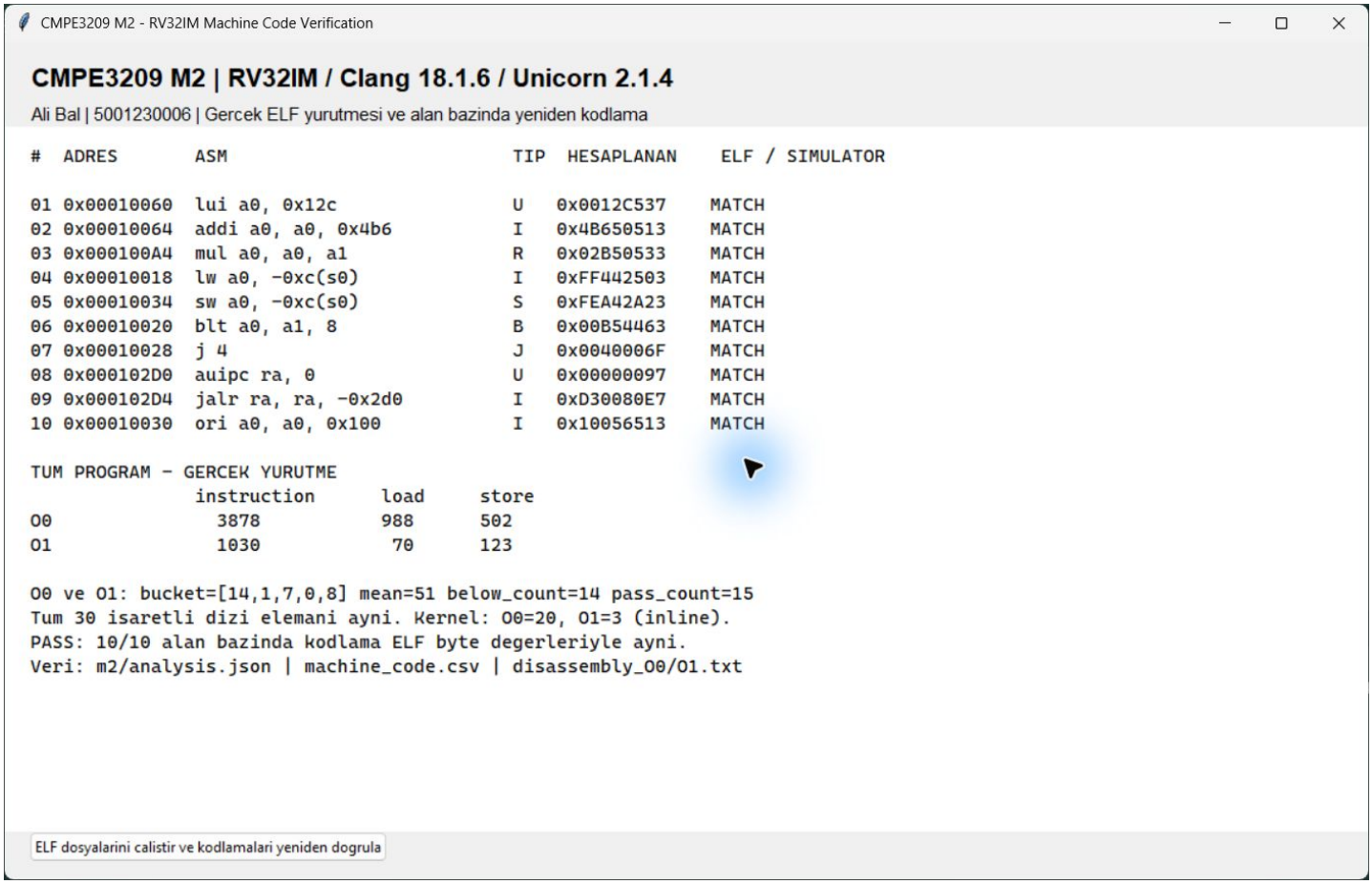
Branch ve fonksiyon çağrısı

blt PC=0x00010020, hedef=0x00010028; offset=8 byte. imm[12:0]=0000000001000; alan dağılımı 0 | 000000 | 0100 | 0, en düşük bit örtük 0'dır. Fall-through adresi 0x00010024'tür. Çağrı: 0x000102D0 auipc ra,0; 0x000102D4 jalr ra,ra,-720. Hedef=(0x000102D0-0x2D0)&~1=0x00010000; dönüş ra=0x000102D8.

Kaynak: RISC-V Unprivileged ISA, RV32I 2.1, instruction formats / integer computational / control transfer bölümleri. https://docs.riscv.org/reference/isa/v20240411/unpriv/rv32.html (erişim: 10 Kasım 2026). RV32M 2.0 çarpma formatı: aynı referansın M bölümü.

Ek-M2.D - Machine code ve yürütme doğrulaması

Alanlardan yeniden üretilen 10 word, bağlı ELF dosyasındaki komutlarla karşılaştırıldı. İki ELF de Unicorn üzerinde main girişinden normal dönüşe kadar çalıştırıldı. Ekran görüntüsü gerçek doğrulama penceresinden alınmıştır. Ödevin izin verdiği simülatör alternatifi kullanılmıştır; bu çıktı objdump olarak adlandırılmamıştır.



Doğrulama dosyaları: m2/analysis.json, m2/machine_code.csv, m2/disassembly_O0.txt, m2/disassembly_O1.txt. Yeniden çalıştırma: python tools/m2_analysis.py. Grafik arayüz: python tools/m2_gui.py.

Model / çalışma koşulları

C99, RV32IM/ILP32, -O0 veya -O1; -g, -fno-sanitize=all, -ffreestanding, -fno-stack-protector, -fno-pic, -nostdlib. Linker: link.ld, giriş main. SP=0x00110000, RA dönüş sentinel'i=0x0003FFF0. BSS sıfırdan başlatılmıştır. Instruction'lar 32 bittir; C uzantısı kullanılmaz.